

House Price Prediction

Import libraries

```
In [1]: import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns

from sklearn.model_selection import train_test_split, RandomizedSearchCV,
KFold, cross_val_score
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import LinearRegression
from sklearn.svm import SVR
from sklearn.tree import DecisionTreeRegressor
from sklearn.ensemble import RandomForestRegressor
from sklearn.metrics import mean_absolute_error, mean_squared_error,
r2_score

pd.set_option('display.float_format', lambda v: f'{v:,.2f}')
```

Load the dataset

```
In [2]: df = pd.read_csv("data.csv")
df.head()
```

Out[2]: Columns 1–8 of 19

	date	price	bedrooms	bathrooms	sqft_living	sqft_lot	floors
0	5/2/2014 0:00	313,000.00	3	1.50	1340	7912	1.50
1	5/2/2014 0:00	2,384,000.00	5	2.50	3650	9050	2.00
2	5/2/2014 0:00	342,000.00	3	2.00	1930	11947	1.00
3	5/2/2014 0:00	420,000.00	3	2.25	2000	8030	1.00
4	5/2/2014 0:00	550,000.00	4	2.50	1940	10500	1.00

Columns 9–15 of 19 · first column repeated

	waterfront	view	condition	sqft_above	sqft_basement	yr_built	yr_renovated
0	0	0	3	1340	0	1955	2005
1	0	4	5	3370	280	1921	0
2	0	0	4	1930	0	1966	0
3	0	0	4	1000	1000	1963	0
4	0	0	4	1140	800	1976	1992

Columns 16–19 of 19 · first column repeated

	street	city	statezip	country
0	18810 Densmore Ave N	Shoreline	WA 98133	USA
1	709 W Blaine St	Seattle	WA 98119	USA
2	26206-26214 143rd Ave SE	Kent	WA 98042	USA
3	857 170th PI NE	Bellevue	WA 98008	USA
4	9105 170th Ave NE	Redmond	WA 98052	USA

```
In [3]: df.shape
```

Out[3]: (4600, 18)

```
In [4]: df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 4600 entries, 0 to 4599
Data columns (total 18 columns):
 #   Column                Non-Null Count  Dtype  
---  -
 0   date                  4600 non-null   object 
 1   price                 4600 non-null   float64
 2   bedrooms              4600 non-null   int64  
 3   bathrooms             4600 non-null   float64
 4   sqft_living           4600 non-null   int64  
 5   sqft_lot              4600 non-null   int64  
 6   floors                4600 non-null   float64
 7   waterfront            4600 non-null   int64  
 8   view                  4600 non-null   int64  
 9   condition             4600 non-null   int64  
10  sqft_above            4600 non-null   int64  
11  sqft_basement         4600 non-null   int64  
12  yr_built              4600 non-null   int64  
13  yr_renovated          4600 non-null   int64  
14  street                4600 non-null   object 
15  city                  4600 non-null   object 
16  statezip              4600 non-null   object 
17  country               4600 non-null   object 
dtypes: float64(3), int64(10), object(5)
memory usage: 647.0+ KB
```

```
In [5]: df.isnull().sum()
```

```
Out[5]: date                0
price                    0
bedrooms                0
bathrooms              0
sqft_living            0
sqft_lot               0
floors                 0
waterfront            0
view                  0
condition              0
sqft_above            0
sqft_basement         0
yr_built              0
yr_renovated          0
street                0
city                  0
statezip              0
country               0
dtype: int64
```

Remove rows where price is 0

```
In [6]: print("rows with price = 0:", (df['price'] == 0).sum())
df = df[df['price'] != 0]
print("rows left:", len(df))
```

```
rows with price = 0: 49
rows left: 4551
```

Remove price outliers using price-per-square-foot instead of raw price

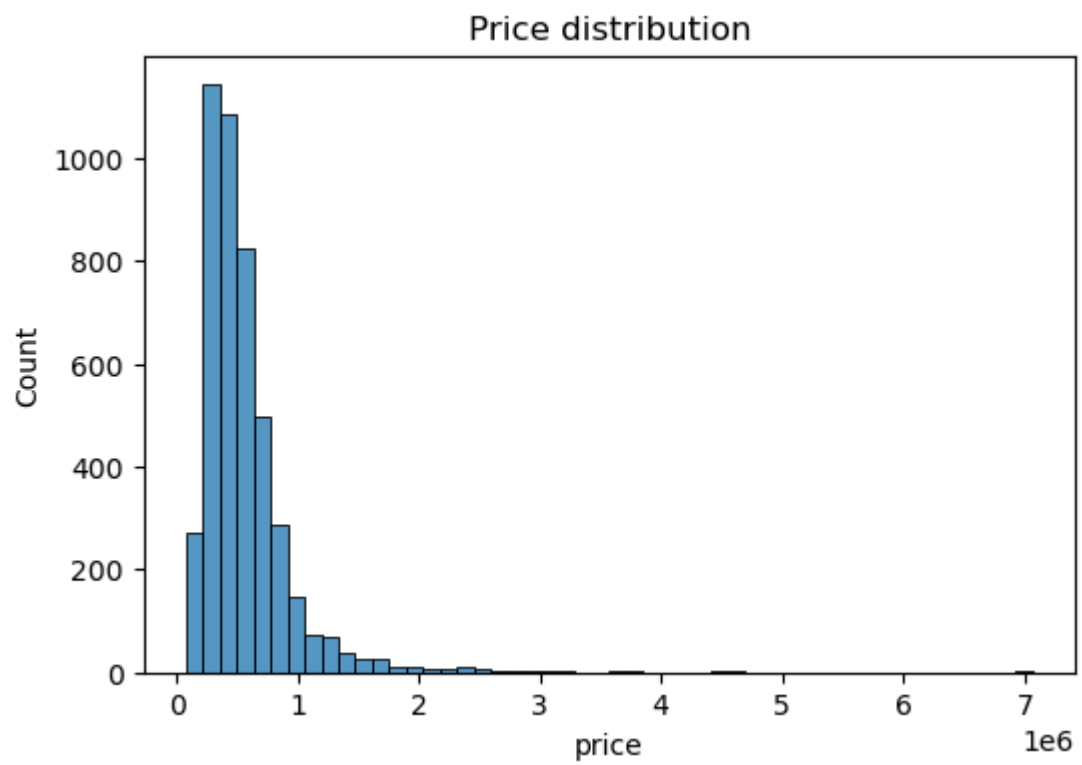
```
In [7]: df['price_per_sqft'] = df['price'] / df['sqft_living']

Q1 = np.log(df['price_per_sqft']).quantile(0.25)
Q3 = np.log(df['price_per_sqft']).quantile(0.75)
IQR = Q3 - Q1
lower = np.exp(Q1 - 3*IQR)
upper = np.exp(Q3 + 3*IQR)

print("removed as price/size outliers:", ((df['price_per_sqft'] < lower) |
(df['price_per_sqft'] > upper)).sum())
df = df[(df['price_per_sqft'] ≥ lower) & (df['price_per_sqft'] ≤ upper)]
df = df.drop(columns=['price_per_sqft'])
print("rows left:", len(df))
```

```
removed as price/size outliers: 5
rows left: 4546
```

```
In [8]: plt.figure(figsize=(6,4))
sns.histplot(df['price'], bins=50)
plt.title("Price distribution")
plt.show()
```



Feature engineering.

```
In [9]: df['date'] = pd.to_datetime(df['date'])
        current_year = df['date'].dt.year

        df['house_age'] = current_year - df['yr_built']
        df['is_renovated'] = (df['yr_renovated'] > 0).astype(int)
        df['years_since_update'] = current_year - np.maximum(df['yr_built'],
        df['yr_renovated'])
        df['has_basement'] = (df['sqft_basement'] > 0).astype(int)

        df['bedrooms'] = df['bedrooms'].replace(0, np.nan)
        df['bathrooms'] = df['bathrooms'].replace(0, np.nan)
        df['sqft_per_bedroom'] = df['sqft_living'] / df['bedrooms']
        df['bath_bed_ratio'] = df['bathrooms'] / df['bedrooms']
        df['living_lot_ratio'] = df['sqft_living'] / df['sqft_lot']
        df['log_sqft_lot'] = np.log1p(df['sqft_lot'])

        df['quality_score'] = df['view'] + df['condition'] + df['waterfront']*3
        df['zipcode'] = df['statezip'].str.split().str[1]

        df.head()
```

Out[9]: Columns 1–8 of 22

	date	price	bedrooms	bathrooms	sqft_living	sqft_lot	floors
0	2014-05-02	313,000.00	3.00	1.50	1340	7912	1.50
1	2014-05-02	2,384,000.00	5.00	2.50	3650	9050	2.00
2	2014-05-02	342,000.00	3.00	2.00	1930	11947	1.00
3	2014-05-02	420,000.00	3.00	2.25	2000	8030	1.00
4	2014-05-02	550,000.00	4.00	2.50	1940	10500	1.00

Columns 9–15 of 22 · first column repeated

	waterfront	view	condition	...	house_age	is_renovated	years_since_update
0	0	0	3	...	59	1	9
1	0	4	5	...	93	0	93
2	0	0	4	...	48	0	48
3	0	0	4	...	51	0	51
4	0	0	4	...	38	1	22

Columns 16–22 of 22 · first column repeated

	has_basement	sqft_per_bedroom	bath_bed_ratio	living_lot_ratio	log_sqft_lot	quality_score	zipcode
0	0	446.67	0.50	0.17	8.98	3	98133
1	1	730.00	0.50	0.40	9.11	9	98119
2	0	643.33	0.67	0.16	9.39	4	98042
3	1	666.67	0.75	0.25	8.99	4	98008
4	1	485.00	0.62	0.18	9.26	4	98052

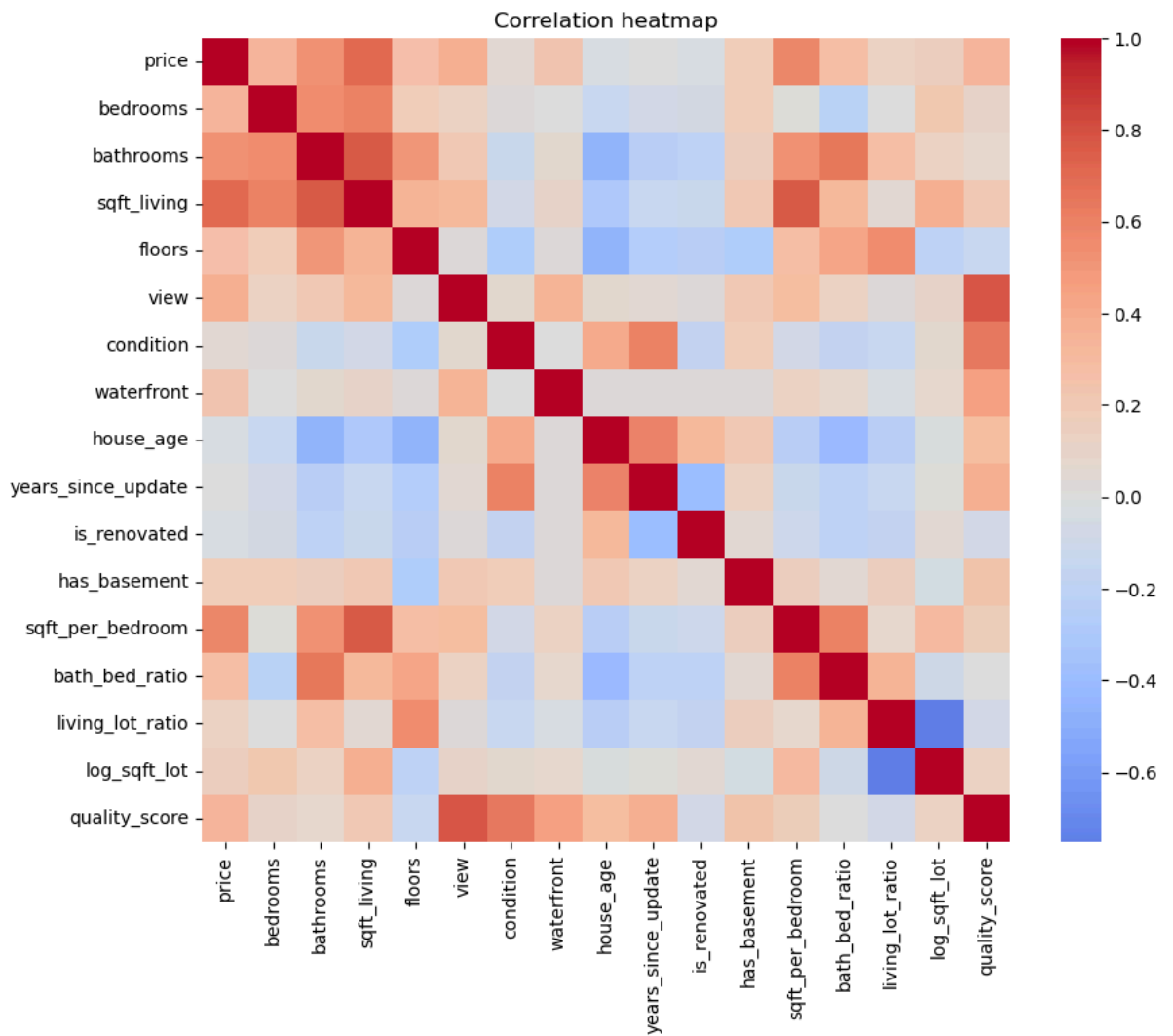
5 rows × 28 columns

Correlation with price

```
In [10]: num_cols =
['price', 'bedrooms', 'bathrooms', 'sqft_living', 'floors', 'view', 'condition', 'waterfront',
        'house_age', 'years_since_update', 'is_renovated', 'has_basement',

'sqft_per_bedroom', 'bath_bed_ratio', 'living_lot_ratio', 'log_sqft_lot', 'quality_score']
plt.figure(figsize=(10,8))
sns.heatmap(df[num_cols].corr(), cmap='coolwarm', center=0, annot=False)
plt.title("Correlation heatmap")
plt.show()

df[num_cols].corr()['price'].sort_values(ascending=False)
```



```
Out[10]: price          1.00
sqft_living      0.70
sqft_per_bedroom 0.58
bathrooms        0.53
view             0.38
bedrooms         0.34
quality_score    0.34
bath_bed_ratio   0.27
floors           0.26
waterfront       0.23
has_basement     0.18
log_sqft_lot     0.16
living_lot_ratio 0.13
condition        0.05
years_since_update -0.00
house_age        -0.03
is_renovated     -0.04
Name: price, dtype: float64
```

Data Preprocessing

Encode city and ZIP code

```
In [11]: feature_cols =
['bedrooms', 'bathrooms', 'sqft_living', 'floors', 'view', 'condition', 'waterfront',
'house_age', 'years_since_update', 'is_renovated', 'has_basement',
'sqft_per_bedroom', 'bath_bed_ratio', 'living_lot_ratio', 'log_sqft_lot', 'quality_score',
'city', 'zipcode']

X = df[feature_cols].copy()
y = df['price']
```

Split the dataset

```
In [12]: X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2,
random_state=42)
X_train.shape, X_test.shape
```

```
Out[12]: ((3636, 18), (910, 18))
```

```

In [13]: log_y_train = np.log1p(y_train)
         global_mean = log_y_train.mean()

         for col in ['city', 'zipcode']:
             stats = log_y_train.groupby(X_train[col]).agg(['mean', 'count'])
             smoothed = (stats['mean'] * stats['count'] + global_mean * 10) /
             (stats['count'] + 10)
             X_train[col + '_enc'] = X_train[col].map(smoothed)
             X_test[col + '_enc'] = X_test[col].map(smoothed).fillna(global_mean)

         X_train = X_train.drop(columns=['city', 'zipcode'])
         X_test = X_test.drop(columns=['city', 'zipcode'])

         X_train = X_train.fillna(X_train.median())
         X_test = X_test.fillna(X_train.median())

         X_train.head()

```

Out[13]: Columns 1–8 of 19

	bedrooms	bathrooms	sqft_living	floors	view	condition	waterfront
540	3.00	2.50	1570	3.00	0	3	0
3946	5.00	2.50	3580	2.00	0	3	0
2097	3.00	3.00	2470	2.00	0	4	0
3279	4.00	2.50	2340	2.00	0	3	0
4332	4.00	2.25	1940	2.00	0	3	0

Columns 9–15 of 19 · first column repeated

	house_age	years_since_update	is_renovated	has_basement	sqft_per_bedroom	bath_bed_ratio	living_lot_ratio
540	4	4	0	0	523.33	0.83	1.10
3946	14	14	0	0	716.00	0.50	0.40
2097	34	34	0	0	823.33	1.00	0.07
3279	21	21	0	0	585.00	0.62	0.27
4332	24	5	1	0	485.00	0.56	0.31

Columns 16–19 of 19 · first column repeated

	log_sqft_lot	quality_score	city_enc	zipcode_enc
540	7.27	3	13.14	13.22
3946	9.10	3	13.28	13.47
2097	10.50	4	13.24	13.19
3279	9.05	3	12.61	12.63
4332	8.73	3	12.59	12.60

```
In [14]: y_train_log = np.log1p(y_train)
```

Linear Regression

```
In [15]: model_LR = LinearRegression()  
model_LR.fit(X_train, y_train_log)
```

```
Out[15]: LinearRegression()
```

In a Jupyter environment, please rerun this cell to show the HTML representation or trust the notebook.

On GitHub, the HTML representation is unable to render, please try loading this page with nbviewer.org.



LinearRegression

[?Documentation for LinearRegression](#)ⁱFitted

Parameters

	fit_intercept	True
	copy_X	True
	tol	1e-06
	n_jobs	None
	positive	False

```
In [16]: y_pred_lr = np.exp1(model_LR.predict(X_test))
```

```
In [17]: mae_lr = mean_absolute_error(y_test, y_pred_lr)  
mse_lr = mean_squared_error(y_test, y_pred_lr)  
rmse_lr = np.sqrt(mse_lr)  
r2_lr = r2_score(y_test, y_pred_lr)  
  
print("Mean Absolute Error:", mae_lr)  
print("Mean Squared Error:", mse_lr)  
print("Root Mean Squared Error:", rmse_lr)  
print("R2-score:", r2_lr)
```

```
Mean Absolute Error: 102043.72052210635  
Mean Squared Error: 103970002076.6853  
Root Mean Squared Error: 322443.7967719108  
R2-score: -0.0133682205016461
```

Support Vector Regression

```
In [18]: scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)

model_svm = SVR(kernel='rbf', C=30, gamma=0.01, epsilon=0.1)
model_svm.fit(X_train_scaled, y_train_log)
```

Out[18]: SVR(C=30, gamma=0.01)

In a Jupyter environment, please rerun this cell to show the HTML representation or trust the notebook.

On GitHub, the HTML representation is unable to render, please try loading this page with nbviewer.org.



SVR

[?Documentation for SVR](#)iFitted

Parameters

	kernel	'rbf'
	degree	3
	gamma	0.01
	coef0	0.0
	tol	0.001
	C	30
	epsilon	0.1
	shrinking	True
	cache_size	200
	verbose	False
	max_iter	-1

```
In [19]: y_pred_svr = np.exp1(model_svm.predict(X_test_scaled))
```

```
In [20]: mae_svr = mean_absolute_error(y_test, y_pred_svr)
mse_svr = mean_squared_error(y_test, y_pred_svr)
rmse_svr = np.sqrt(mse_svr)
r2_svr = r2_score(y_test, y_pred_svr)

print("Mean Absolute Error:", mae_svr)
print("Mean Squared Error:", mse_svr)
print("Root Mean Squared Error:", rmse_svr)
print("R2-score:", r2_svr)
```

```
Mean Absolute Error: 90849.81883614152
Mean Squared Error: 31089407260.90691
Root Mean Squared Error: 176321.88537134835
R2-score: 0.6969797375862415
```

Decision Tree

```
In [21]: model_dt = DecisionTreeRegressor(
            max_depth=8,
            min_samples_split=10,
            min_samples_leaf=4,
            random_state=42
        )
        model_dt.fit(X_train, y_train_log)
```

```
Out[21]: DecisionTreeRegressor(max_depth=8, min_samples_leaf=4, min_samples_split=10,
                                random_state=42)
```

In a Jupyter environment, please rerun this cell to show the HTML representation or trust the notebook.

On GitHub, the HTML representation is unable to render, please try loading this page with nbviewer.org.



DecisionTreeRegressor

[?Documentation for DecisionTreeRegressor](#)

Parameters

	criterion	'squared_error'
	splitter	'best'
	max_depth	8
	min_samples_split	10
	min_samples_leaf	4
	min_weight_fraction_leaf	0.0
	max_features	None
	random_state	42
	max_leaf_nodes	None
	min_impurity_decrease	0.0
	ccp_alpha	0.0
	monotonic_cst	None

```
In [22]: y_pred_dt = np.expm1(model_dt.predict(X_test))
```

```
In [23]: mae_dt = mean_absolute_error(y_test, y_pred_dt)
mse_dt = mean_squared_error(y_test, y_pred_dt)
rmse_dt = np.sqrt(mse_dt)
r2_dt = r2_score(y_test, y_pred_dt)

print("Mean Absolute Error:", mae_dt)
print("Mean Squared Error:", mse_dt)
print("Root Mean Squared Error:", rmse_dt)
print("R2-score:", r2_dt)
```

```
Mean Absolute Error: 107027.98497187652
Mean Squared Error: 32120111792.13713
Root Mean Squared Error: 179220.84642177407
R2-score: 0.686933732047977
```

Random Forest

```
In [24]: params = {  
    "n_estimators": [200, 300, 400, 500],  
    "max_depth": [None, 20, 30, 40],  
    "min_samples_split": [2, 5, 10],  
    "min_samples_leaf": [1, 2, 4],  
    "max_features": [0.5, 0.7, 1.0, "sqrt"],  
}  
  
model_rf = RandomizedSearchCV(  
    RandomForestRegressor(random_state=42),  
    param_distributions=params,  
    n_iter=30,  
    cv=5,  
    scoring='r2',  
    random_state=42,  
    n_jobs=-1  
)  
model_rf.fit(X_train, y_train_log)
```

```
Out[24]: RandomizedSearchCV(cv=5, estimator=RandomForestRegressor(random_state=42),
                             n_iter=30, n_jobs=-1,
                             param_distributions={'max_depth': [None, 20, 30, 40],
                                                    'max_features': [0.5, 0.7, 1.0, 'sqrt'],
                                                    'min_samples_leaf': [1, 2, 4],
                                                    'min_samples_split': [2, 5, 10],
                                                    'n_estimators': [200, 300, 400, 500]},
                             random_state=42, scoring='r2')
```

In a Jupyter environment, please rerun this cell to show the HTML representation or trust the notebook.

On GitHub, the HTML representation is unable to render, please try loading this page with nbviewer.org.



RandomizedSearchCV

[?Documentation for RandomizedSearchCV](#)

Parameters

	estimator	RandomForestR...ndom_state=42)
	param_distributions	{'max_depth': [None, 20, ...], 'max_features': [0.5, 0.7, ...], 'min_samples_leaf': [1, 2, ...], 'min_samples_split': [2, 5, ...], ...}
	n_iter	30
	scoring	'r2'
	n_jobs	-1
	refit	True
	cv	5
	verbose	0
	pre_dispatch	'2*n_jobs'
	random_state	42
	error_score	nan
	return_train_score	False



best_estimator_: RandomForestRegressor

```
RandomForestRegressor(max_depth=40, max_features=0.5, min_samples_leaf=2,
                       n_estimators=500, random_state=42)
```



RandomForestRegressor

[?Documentation for RandomForestRegressor](#)

Parameters

	n_estimators	500
	criterion	'squared_error'

	max_depth	40
	min_samples_split	2
	min_samples_leaf	2
	min_weight_fraction_leaf	0.0
	max_features	0.5
	max_leaf_nodes	None
	min_impurity_decrease	0.0
	bootstrap	True
	oob_score	False
	n_jobs	None
	random_state	42
	verbose	0
	warm_start	False
	ccp_alpha	0.0
	max_samples	None
	monotonic_cst	None

```
In [25]: print(model_rf.best_params_)
print("best CV R2 (on training data):", model_rf.best_score_)

{'n_estimators': 500, 'min_samples_split': 2, 'min_samples_leaf': 2,
 'max_features': 0.5, 'max_depth': 40}
best CV R2 (on training data): 0.8187170721664006
```

```
In [26]: best_rf = model_rf.best_estimator_
y_pred_rf = np.expm1(best_rf.predict(X_test))
```

```
In [27]: mae_rf = mean_absolute_error(y_test, y_pred_rf)
mse_rf = mean_squared_error(y_test, y_pred_rf)
rmse_rf = np.sqrt(mse_rf)
r2_rf = r2_score(y_test, y_pred_rf)

print("Mean Absolute Error:", mae_rf)
print("Mean Squared Error:", mse_rf)
print("Root Mean Squared Error:", rmse_rf)
print("R2-score:", r2_rf)
```

```
Mean Absolute Error: 83583.79216757433
Mean Squared Error: 21487766736.546806
Root Mean Squared Error: 146587.0619684657
R2-score: 0.7905643983318547
```

Compare all four models

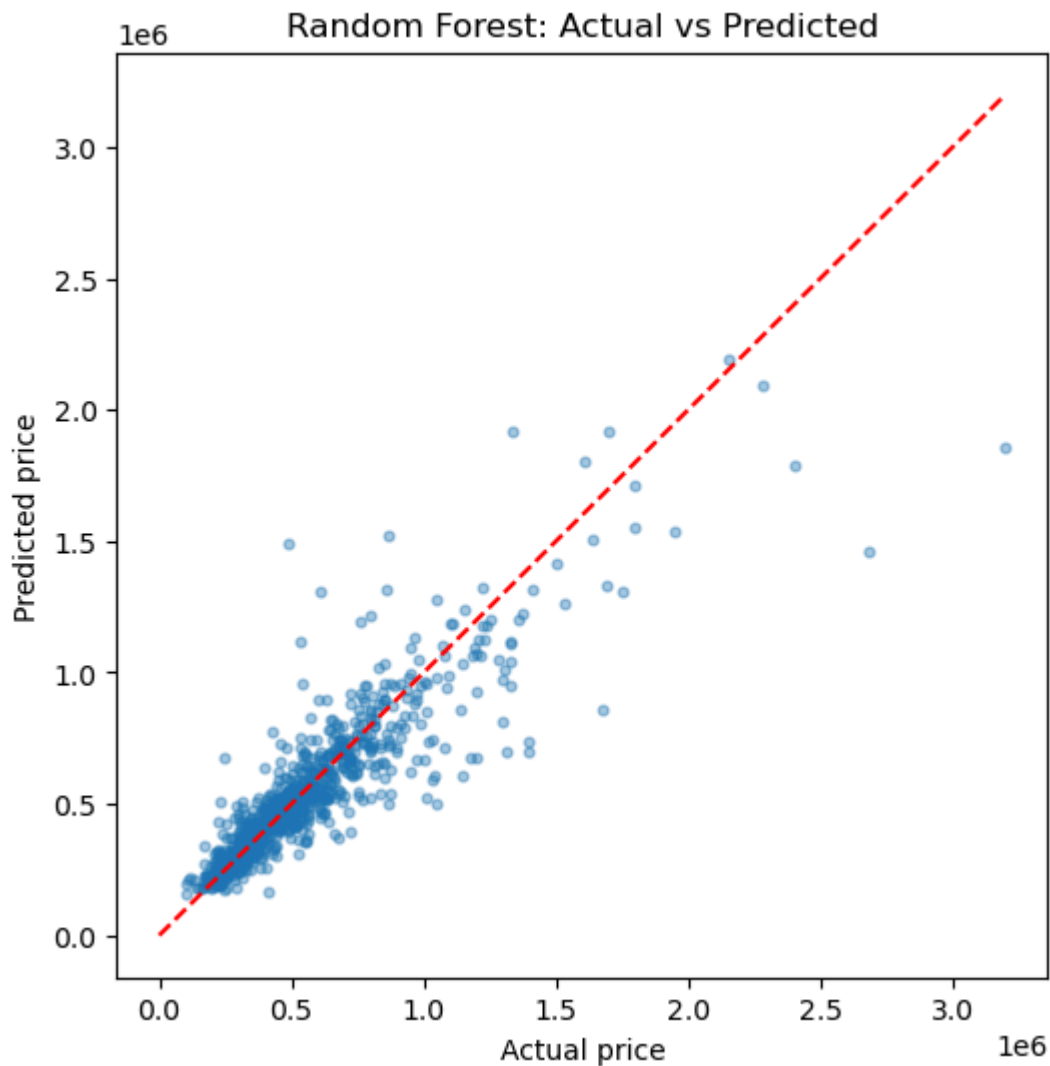
```
In [28]: results = pd.DataFrame({
    'Model': ['Linear Regression', 'SVR', 'Decision Tree', 'Random Forest'],
    'MAE': [mae_lr, mae_svr, mae_dt, mae_rf],
    'MSE': [mse_lr, mse_svr, mse_dt, mse_rf],
    'RMSE': [rmse_lr, rmse_svr, rmse_dt, rmse_rf],
    'R2': [r2_lr, r2_svr, r2_dt, r2_rf]
})
results.sort_values('R2', ascending=False)
```

Out[28]:

	Model	MAE	MSE	RMSE	R2
3	Random Forest	83,583.79	21,487,766,736.55	146,587.06	0.79
1	SVR	90,849.82	31,089,407,260.91	176,321.89	0.70
2	Decision Tree	107,027.98	32,120,111,792.14	179,220.85	0.69
0	Linear Regression	102,043.72	103,970,002,076.69	322,443.80	-0.01

Actual vs predicted price

```
In [29]: plt.figure(figsize=(6,6))
plt.scatter(y_test, y_pred_rf, alpha=0.4, s=12)
lims = [0, max(y_test.max(), y_pred_rf.max())]
plt.plot(lims, lims, 'r--')
plt.xlabel("Actual price")
plt.ylabel("Predicted price")
plt.title("Random Forest: Actual vs Predicted")
plt.show()
```



5-fold cross-validation check (on training data only)

```
In [30]: cv = KFold(n_splits=5, shuffle=True, random_state=42)
cv_scores = cross_val_score(best_rf, X_train, y_train_log, cv=cv,
scoring='r2')
print("Random Forest CV R2 scores:", cv_scores)
print("Average:", cv_scores.mean(), " Spread:", cv_scores.std())
```

```
Random Forest CV R2 scores: [0.82088309 0.81134223 0.82296943 0.8408343
0.80667168]
```

```
Average: 0.820540147553349 Spread: 0.011788194977904695
```